

# Worksheet SOLUTIONS: RDF, RDFS & OWL

Foundations of Artificial Intelligence – SS 2026 *University of Bremen,  
Institute for Artificial Intelligence*

---

## Part 1 – RDF Triples

### 1.1 Turtle triples:

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

ex:MarieCurie rdf:type ex:Person .
ex:MarieCurie ex:bornOn "1867-11-07"^^xsd:date .
ex:MarieCurie ex:discovered ex:Radium .
ex:Radium rdf:type ex:ChemicalElement .
ex:MarieCurie owl:sameAs <http://dbpedia.org/resource/Marie_Curie> .
```

**1.2 (c) Object only.** Literals are just values; they cannot be subjects (no outgoing edges) and cannot be predicates (too meaningless). See Triples V table in the lecture.

### 1.3 Shorthand with semicolon:

```
ex:Paris ex:isA ex:City ;
        ex:locatedIn ex:France .
```

**1.4** Blank nodes represent anonymous resources useful for complex structured values that do not need their own URI. Example — “*Alice weighs 62 kg*”:

```
ex:Alice ex:weighs [
    ex:value 62 ;
    ex:unit ex:kg
] .
```

---

## Part 2 – RDFS

**2.1** Inference chain: 1. `:Michaela rdf:type :Teacher` — Michaela is a Teacher. 2. `:Teacher rdfs:subClassOf :Person` → Michaela is also a Person. 3. `:Person rdfs:subClassOf :LivingBeing` → Michaela is also a LivingBeing.

RDFS entailment propagates `rdf:type` through the full subclass hierarchy.

### 2.2 Completed snippet:

```

:Professor rdf:type rdfs:Class .
:teachesCourse rdf:type rdfs:Property .
:teachesCourse rdfs:domain :Professor .
:teachesCourse rdfs:range :Course .
:teachesCourse rdfs:subPropertyOf :instructs .

```

### 2.3 Two RDFS limitations:

1. **Class disjointness** — RDFS cannot state that a City cannot simultaneously be a Country. This requires `owl:disjointWith`.
2. **Property cardinality** — RDFS cannot express that every country has *exactly one* capital. This requires OWL restrictions (`owl:cardinality`).

---

## Part 3 – OWL Properties

### (a) `owl:SymmetricProperty`

```

:siblingOf a owl:SymmetricProperty .
ex:Alice :siblingOf ex:Bob .
# ex:Bob :siblingOf ex:Alice .

```

### (b) `owl:TransitiveProperty`

```

:tallerThan a owl:TransitiveProperty .
ex:Alice :tallerThan ex:Bob .
ex:Bob :tallerThan ex:Carol .
# ex:Alice :tallerThan ex:Carol .

```

### (c) `owl:FunctionalProperty`

```

:hasBiologicalMother a owl:FunctionalProperty .
# A person can be linked to at most one biological mother.

```

### (d) `owl:InverseFunctionalProperty`

```

:bornIn a owl:InverseFunctionalProperty .
# If ex:A :bornIn ex:Place . and ex:B :bornIn ex:Place .
# ex:A owl:sameAs ex:B .

```

### (e) `owl:inverseOf`

```

:worksFor owl:inverseOf :employerOf .
ex:Alice :worksFor ex:UniBremen .
# ex:UniBremen :employerOf ex:Alice .

```

---

## Part 4 – OWL Restrictions

### 4.1

(i) Every Pizza has exactly 1 base:

```
:Pizza owl:equivalentClass [  
  rdf:type owl:Restriction ;  
  owl:cardinality "1"^^xsd:int ;  
  owl:onProperty :hasBase  
] .
```

(ii) VeganPizza — all toppings are VeganTopping:

```
:VeganPizza rdfs:subClassOf :Pizza ;  
  rdfs:subClassOf [  
    rdf:type owl:Restriction ;  
    owl:onProperty :hasTopping ;  
    owl:allValuesFrom :VeganTopping  
  ] .
```

(iii) MeatLover — at least one MeatTopping:

```
:MeatLover rdfs:subClassOf :Pizza ;  
  rdfs:subClassOf [  
    rdf:type owl:Restriction ;  
    owl:onProperty :hasTopping ;  
    owl:someValuesFrom :MeatTopping  
  ] .
```

**4.2** :capitalOf is an owl:InverseFunctionalProperty, meaning its *subject* is uniquely determined by its *object*. Both :Berlin and :Hauptstadt are the value of :capitalOf :Germany. Therefore the reasoner infers:

```
:Berlin owl:sameAs :Hauptstadt .
```

---

## Part 5 – Spotting Mistakes

(a) **Mistake:** ex:hasName is declared as owl:ObjectProperty, but it is used with a string literal "Robert". Object properties must link to individuals (URIs), not literals. **Fix:** change to owl:DatatypeProperty.

```
ex:hasName rdf:type owl:DatatypeProperty .  
ex:Bob ex:hasName "Robert" .
```

(b) **Mistake:** Circular subclass definitions. With ex:Doctor rdfs:subClassOf ex:Person and ex:Person rdfs:subClassOf ex:Doctor, a reasoner infers that Doctor and Person are equivalent (owl:equivalentClass). This is almost certainly unintended and may cause unexpected inferences. **Fix:** Remove one direction, or explicitly declare equivalence if intended.

(c) **Mistake:** Declaring :Cat owl:disjointWith :Animal while :Whiskers rdf:type :Cat creates a contradiction, because every individual is implicitly a

member of `owl:Thing`, and any class is a subclass of `owl:Thing` — but more critically, a cat *is* an animal in any sensible domain model. A reasoner will derive an inconsistency because `:Whiskers` would have to be both a `:Cat` and, through domain knowledge, an `:Animal`, yet the two are declared disjoint. **Fix:** Make `:Cat` `rdfs:subClassOf :Animal` instead, and use `owl:disjointWith` only between truly non-overlapping classes.

---

## Part 6 – Python: Simple RDF Graph Builder

```
from rdflib import Graph, Namespace, Literal, RDF, URIRef
from rdflib.namespace import XSD

EX = Namespace("http://example.org/")
g = Graph()
g.bind("ex", EX)

# 6.1 - Build the graph
g.add((EX.Alice, RDF.type, EX.Person))
g.add((EX.Alice, EX.knows, EX.Bob))
g.add((EX.Bob, RDF.type, EX.Person))
g.add((EX.Alice, EX.hasAge, Literal(30, datatype=XSD.integer)))
g.add((EX.Bob, EX.worksAt, EX.UniBremen))
g.add((EX.UniBremen, RDF.type, EX.University))

print(g.serialize(format="turtle"))

# 6.2 - Query: all Person instances
print("\nAll persons:")
for subj, pred, obj in g.triples((None, RDF.type, EX.Person)):
    print(" -", subj)

# 6.3 - Serialize and reload
g.serialize(destination="knowledge_base.ttl", format="turtle")
g2 = Graph()
g2.parse("knowledge_base.ttl", format="turtle")
assert len(g) == len(g2), "Round-trip failed!"
print(f"\nRound-trip OK - {len(g2)} triples loaded.")
```

Expected output (triples):

```
ex:Alice a ex:Person ;
        ex:hasAge 30 ;
        ex:knows ex:Bob .
ex:Bob   a ex:Person ;
        ex:worksAt ex:UniBremen .
```

ex:UniBremen a ex:University .

All persons:

- <http://example.org/Alice>
- <http://example.org/Bob>

Round-trip OK - 6 triples loaded.

---

*End of solutions.*